

Binary Hacking: Cheats for Game Boy Wars 2

Thomas Hetebrüg¹ Leah Ludwig¹ Julius Hamel¹

¹Heidelberg University

Motivation and Challenges

We chose the GameBoy Color game Game Boy Wars 2 (GBW2) as a topic for the practical. Our goal was to get an understanding of key game mechanics and to write cheats based on the mechanics we identified. We wanted to write a user-friendly framework where GBW2 can be played in an emulator and the cheats can dynamically be applied in the same place.

We specifically chose GBW2 as a relatively unknown game with little available sources to start this practical from scratch. The turn based nature of the strategy game GBW2 gave us an easy entryway to analyze the memory state in the beginning, but also made it quite challenging to keep track of instructions and memory blocks being used by both players.

Since none of us had any greater experience with assembly code and reverse engineering it took us quite some time to grasp the topic. Especially gaining understanding of the combat damage calculations was complex. While the calculation is nothing extraordinary on a high level, the low level implementations of shift-and-add multiplication and shift-and-subtract divisions required time and effort to spot and understand in the game loop.

Game Boy Wars 2 Gameplay

GBW 2 is a turn-based strategy game for one or two players. In the singleplayer mode you play against a computer controlled enemy. Alternatively you can opt for the multiplayer mode where two players can play against each other through a Game Link Cable connection or through pass-and-play on the same device.

You can win the game by capturing your enemies 'Command Base' structure or kill all of their units in combat.

Each player can recruit one of 21 different unit types, such as Infantry units, tanks or submarines in unit specific structures, like 'Commercial', 'Ship Yard' or 'Airports'. Each unit has a different price and combat stats. Units can have up to two weapons, which have different damage in combat. They also have different amounts of HP and 'Ammunition'. A unit loses 'Ammunition' while traveling or fighting, but they can be resupplied by supporting units. Additionally the different units also have different movement ranges and might not be able to move onto every tile type.

Players earn money each round, which is increased by 'Industrial' structures. These structures can be conquered by the opposing team.

GBW2 comes with 24 different maps to play and has a built-in map editor tool, where you can create your own maps.

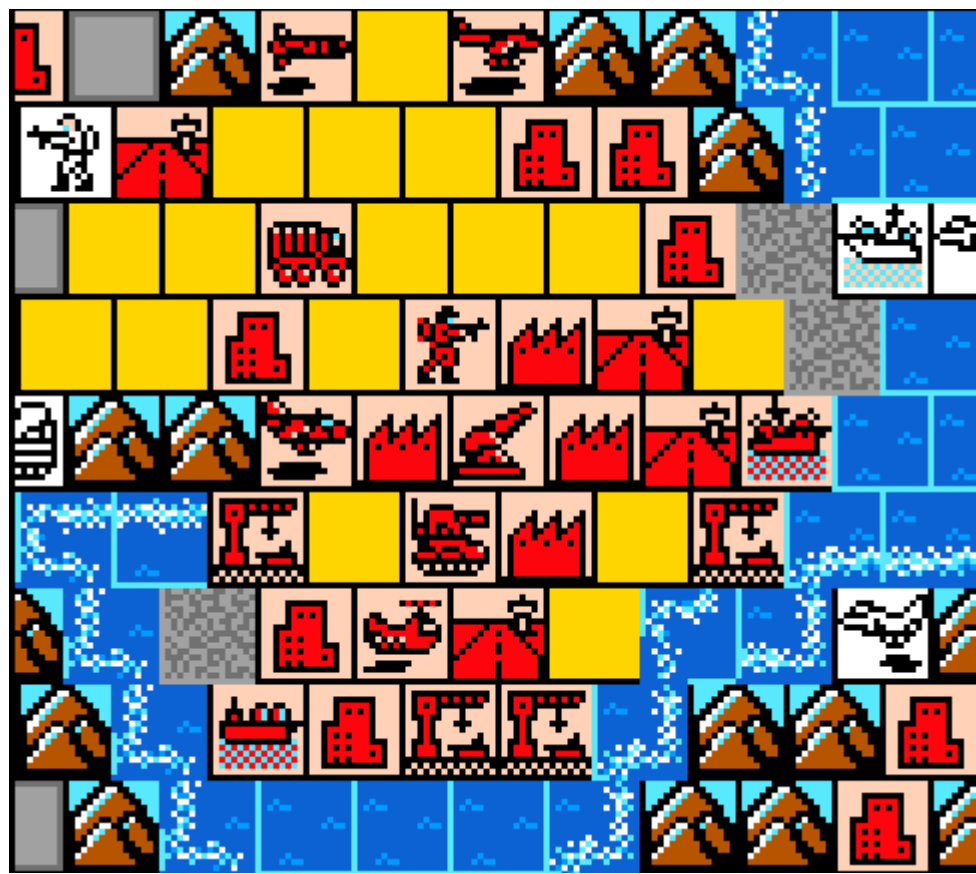


Figure 1. A screenshot from the GBW2 intro

Methodology and Concept

In the beginning while having no grasp for the game loop we simply started by searching the RAM memory for values that correlate with a value of interest in the game, for example a unit's HP values. We would then verify and investigate these potential memory addresses by setting read and write breakpoints in an emulator and looking how and when these values were modified in detail.

After gaining more knowledge about the structure of GBW2, we were also able to identify the use and structure of memory blocks more easily. One good example for this are the memory addresses of structures, which come right after the RAM block of combat units. After learning and understanding about the units' memory, we were able to understand the concept of the structure memory much faster.

We wrote a Python framework based on the emulator pyboy, which also has a memory manipulation API, with which can be used to run GBW2 and select wanted cheats for either team from a graphical interface.

Most of our cheats in this framework are based on pyboy hooks, which trigger when the program counter (PC) reaches a specific instruction. We chose to go for this approach instead of static assembly manipulation to make it easier to enable a cheat for a single team.

While there are a few functions and procedures which get called in the same team order every time where we could have injected our own modified assembly functions at other times, it is the exact same instruction being called for both teams, that loads different values based on addresses in the HL registers. Since we were unable to effectively handle such cases, we opted for pyboy hooks and high-level functions, such as if-else, to check if a cheat should be applied or not.

Unit and Structure memory

After buying a unit, their stats get saved in 16-byte blocks in the memory addresses C980 to CE9F, with the units of the red team being saved in C980-CC09 and CC10-CE9F containing the white units.

C980:	00 03 08 03	00 00 32 00	63 63 49 00	28 00 00 00
C990:	7F 7F 7F 7F	7F 7F 7F 7F	7F 7F 7F 7F	7F 7F 7F 7F
C9A0:	7F 7F 7F 7F	7F 7F 7F 7F	7F 7F 7F 7F	7F 7F 7F 7F

Figure 2. A red 'Infantry' in memory

The first byte depicts the unit type. There are 11 different unit types with red units being represented by even and white units by odd numbers. The second and third byte represent the column and the row of the tile the unit is standing on. Additionally, the tile type is saved in the seventh byte. The table below shows some of the possible tile types.

Tile type	Byte
Red Industrial	32
White Industrial	33
Grass	3D
Forest	3F
Mountain	40

The fourth byte depicts the unit's 'exhaustion' state. Units are only allowed one move per turn. If a unit is allowed to move, this byte is 02; if it has already moved or has just been recruited this turn, it is 03.

The third block of bytes contains a unit's stats, with the ninth byte saving the unit's HP, the tenth byte representing the ammunition of a unit, which gets decreased through movement or combat, and bytes eleven and twelve being weapon stats, which are used to calculate the combat damage.

The last four bytes seem to be static for all units of the different teams.

The memory addresses containing the stats for structures are located right behind the unit memory addresses, starting at address CEAO. The stats for one structure are represented in a four-byte block.

CEA0:	3A 07 04 14	32 02 06 14	32 03 06 14	3B 06 06 14
CEB0:	3B 08 06 14	33 0B 06 14	33 0C 06 14	32 02 07 14

Figure 3. Eight structures on map 'Devilman' in memory

The first byte represents the structure type. The table below shows all tile types of the map 'Devilman'.

Structure Type	Byte
Red Command Base	3A
White Command Base	3B
Red Industrial	32
White Industrial	33
Red Commercial	30
White Commercial	31

The second and third bytes are the tile coordinates, similar as in unit memory. The fourth byte contains the structure HP, which is divided by 10 compared to its displayed value.

Combat Loop

Whenever a unit is selected, their memory is copied from the memory addresses described above to another section in the RAM, starting at C4D3. Once another unit is selected to be attacked, their memory is copied to C517 - C526. When combat is initialized, the game calculates the damage for both units and updates the previously mentioned RAM sections. This can be described with the following equations, where unitTypeValue and tileTypeValue are loaded from the ROM, depending on the unit type and tile type.

$$\text{newHP} = \text{currentHP} - (\text{baseDamage} - \text{damageReduction}) \quad (1)$$

$$\text{baseDamage} = \left\lfloor \frac{\text{unitTypeValue} * \text{opposingUnitHP}}{99} \right\rfloor + \begin{cases} 5 & \text{for attacked team} \\ 0 & \text{otherwise} \end{cases} \quad (2)$$

$$\text{damageReduction} = \left\lfloor \frac{\text{tileTypeValue} * \text{baseDamage}}{100} \right\rfloor \quad (3)$$

The current HP for the red team gets loaded into C60C, and the new HP gets loaded into C611. The white team memory addresses are C614 for the current HP and C619 for the new HP. These values then get transformed into the displayed HP in the range 1-10, with the following calculation:

$$\text{displayHP} = \left\lfloor \frac{\text{memoryHP}}{10} \right\rfloor + 1 \quad (4)$$

The game then loops the combat animation for both teams, decreasing the current HP until it is the same as the new HP. For every decrease, one individual of the unit dies. After the combat is done, the unit's new stats get loaded back to their base memory block.

Results

As mentioned earlier, we created a framework to run the game and select the cheats through a GUI.



Figure 4. A screenshot of our framework

The full list of cheats we achieved to write is:

- Invincibility**
Our Invincibility cheat makes units invincible. They do not lose any HP in combat, and no units die in combat animation, due to setting the new HP value the same as the current HP value after their initialization.
- Money Locking**
When Money Locking is active, a team's money does not get decreased after buying a new unit. We achieved this by comparing the state of a team's money every tick in the emulator and reverting it back up to the previous state if it has been decreased.
- Infinite Moves**
The Infinite Moves cheat allows units to move multiple times in one turn; this includes attacking multiple times. The cheat reverts a bit-set operation that changes the fourth byte of the unit's memory block from 02 to 03. Additionally, when activating this cheat, all existing units in memory are checked, and any 'exhausted' state is overwritten.
- Instant Capture**
Instant Capture makes units capture enemy structures in one turn, even if they have more HP than the unit's structure damage. The normal structure damage is calculated the same way as the displayed HP of a unit. When the cheat is active, the full current HP is being subtracted from the structure instead of the unit's calculated structure damage.
- One-Hit Kill**
The One-Hit Kill cheat is the opposite of invincibility and makes units kill every enemy unit in one attack. To achieve this, we set a hook on the instruction in the combat loop that sets the new HP, and this hook sets the new HP to zero. Since this calculation happens twice every turn, we implemented a boolean flag that only makes this cheat trigger on the second calculation loop execution, where the registers C611 and C619 strictly represent the red and white team respectively.
- Money Manipulation**
Our Money Manipulation cheats allow the player to manually increase or decrease their own or the enemy's money by an arbitrary input or pre-set amount. The money is capped at 99,999 and is represented by three bytes each for the two teams. To manipulate the money, we simply calculate the new money and load it accordingly into the three bytes.



Figure 5. A screenshot of the Money Manipulation GUI

Future Work

For future work, it would be very interesting to analyze how the computer controls a player and makes gameplay decisions. Additionally, attempting to add new functions in free memory space or with Python API support, like 'capturing' defeated enemy units by transferring their memory into your team's unit memory, would be a nice idea.